Quick Sort

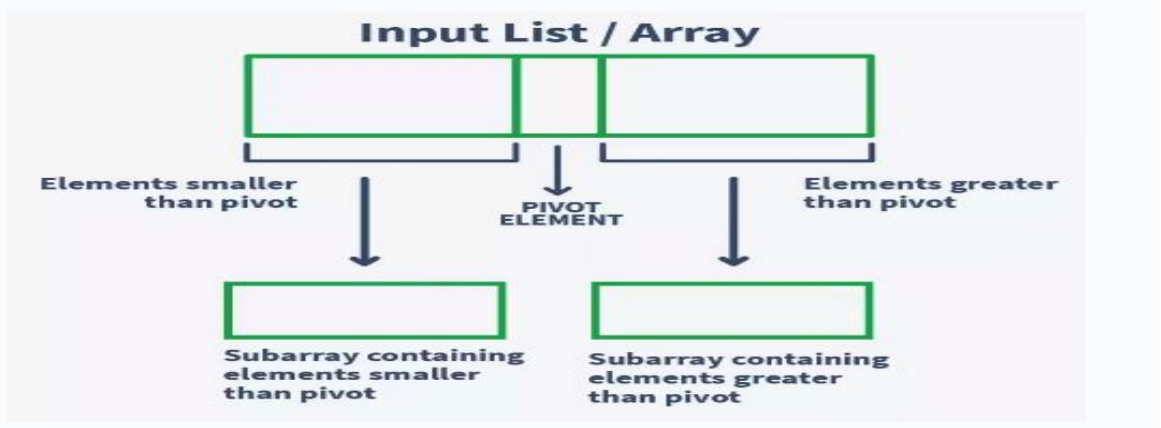
- Partition exchange sort
- Follows divide and conquer strategies
- A problem is divided into smaller problems and so on
- In-place sort so no additional space is required for sorting.
- Faster than all other algorithms
- Very efficient because exchange occurs between elements that are far apart and so less exchanges are needed to place an element in its final position.
- Implemented in both iterative and Recursion modes.
- The Initial step of the Quick Sort algorithm is to select the pivot element and then rearrange the elements around the pivot element. A pivot can be any element in the array or a pivot can be the first or last element of the array or it could either be the median of the array.

What is Quick sort?

Quick sort also known as partition-exchange sort, is an in-place sorting algorithm. It is a divide-and-conquer algorithm that **works on the idea of selecting a pivot element and dividing the array into two subarrays around that pivot.**

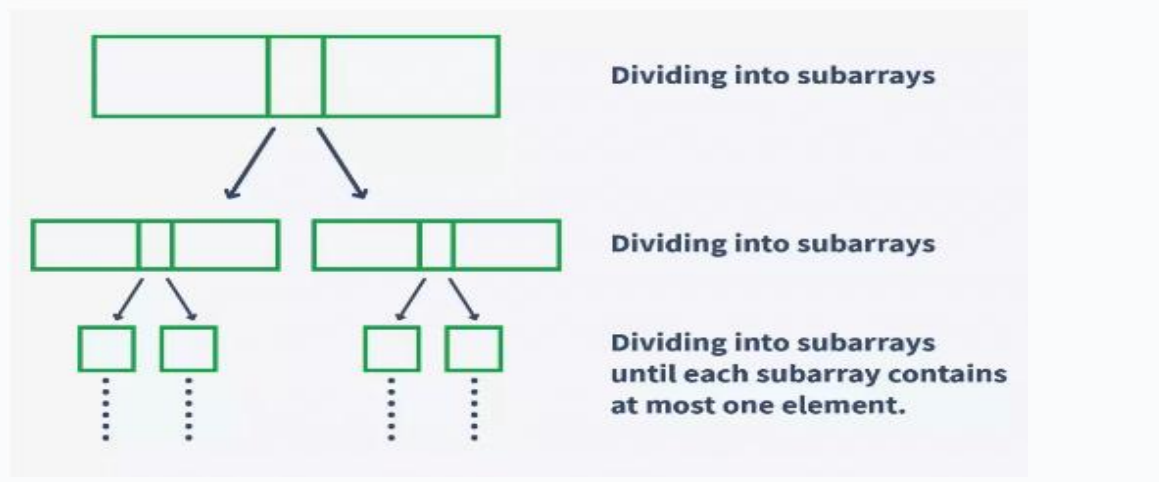
Any element from the list is considered as pivot element, as the first element, the last element, or any random element from the array.

In quick sort, after selecting the pivot element, the array is split into two subarrays. One subarray contains elements smaller than the pivot element, and the other subarray contains elements greater than the pivot element.



At every iteration of quick sort, the chosen pivot element is placed at its correct position that it should acquire in the sorted array. This makes sure that each chosen pivot element is at its correct sorted position.

This process of selecting the pivot element and dividing the array into subarrays is carried out recursively until all the elements in the array are sorted. In other words, each subarray is divided further until each subarray consists of only one element. All these subarrays are then combined to form a single sorted array. Since we know before partitioning of the array, the pivot is placed at its correct position, dividing the array to one single element places each element at its correct sorted position, and thus, combining all of them gives us a sorted array.



Time Complexity

Case	Time Complexity
Best Case	O(nlogn)
Average Case	O(nlogn)
Worst Case	$O(n^2)$

Space Complexity

Space Complexity	O(n*logn)
Stable	NO

Advantages of Quicksort

- It is a divide-and-conquer algorithm that makes it easier to solve problems.
- It is efficient on large data sets.
- It does not require any extra memory, unlike merge sort. This makes Quicksort faster since it does not require allocation and deallocation of memory.
- Less time and space complexity.
- It has a low overhead, as it only requires a small amount of memory to function.
- It is Cache Friendly as we work on the same array to sort and do not copy data to any auxiliary array.
- Most implementation is in randomized order and quick sort works better for randomized version.

Disadvantages of Quicksort

- It has a worst-case time complexity of O(n2), which occurs when the pivot is chosen poorly.
- It is not a good choice for small data sets.
- It is not a stable sort, meaning that if two elements have the same key, their relative order will not be preserved in the sorted output in case of quick sort, because here we are swapping elements according to the pivot's position
- Quicksort requires random access of elements which is not possible in a linked list. For these reasons, quicksort is not preferable in a linked list.

Applications of Quicksort in Data Structure

- Used for sorting in commercial computing since it is the fastest.
- Most algorithms that are efficiently developed, use priority queues. This makes the implementation of quicksort easy.
- Quicksort is a cache-friendly algorithm.
- It is used in event-driven simulations and to implement primitive type methods.

Python Function to sort data using quick Sort

```
def quickSort(a,low,high):
    if low<high:
        pivot=partition(a,low,high)
        quickSort(a,low,pivot-1)
        quickSort(a,pivot+1,high)

def partition(a,low,high):
    p=a[low]
    i=low+1
    j=high
    while True:
        while i<=j and a[i]<=p:
            i=i+1
        while i<=j and a[j]>=p:
            j=j-1
        if i<=j:
            a[i],a[j]=a[j],a[i]
        else:
            break
    a[low],a[j]=a[j],a[low]
    return j

a=[5,8,1,2,6,3,9]
quickSort(a,0,len(a)-1)
print(a)
```